

Attention in Neural Machine Translation: A Short Tutorial

Passage Sample Documents¹

Abstract

Neural machine translation maps a sentence in one language to a sentence in another with a single trainable network. Early encoder–decoder systems compressed the whole source sentence into one fixed-length vector, which made long sentences difficult to translate. Attention mechanisms removed this bottleneck by letting the decoder consult every source position at every step. This tutorial reviews how attention is computed, why the Transformer relies on it exclusively, and how translation quality is usually measured. We keep the notation consistent across sections and point out practical details that are easy to overlook, such as scaling, masking, and subword segmentation.

1 Introduction

Translating a sentence requires deciding which parts of the source are relevant to each word of the output. Statistical systems made this decision explicit through word alignments. Neural systems instead learn it from data, and the component that performs this soft alignment is called attention.

The encoder–decoder architecture [7, 2] was the first neural design to translate whole sentences end to end. An encoder reads the source sentence and a decoder generates the translation one token at a time. In its original form the encoder summarized the entire sentence in a single vector. This worked surprisingly well for short sentences, but quality dropped as sentences grew longer, because a fixed-length vector cannot hold an arbitrary amount of information.

Attention [1] addressed this limitation. Instead of one summary vector, the decoder receives a different weighted combination of encoder states at every step. The weights are computed from the current decoder state, so the model can focus on different source words as the translation proceeds. Later work showed that attention alone, without any recurrence, is sufficient for state-of-the-art translation [8].

The rest of this tutorial is organized as follows. Section 2 introduces the encoder–decoder model. Section 3 defines attention and its common scoring functions. Section 4 describes self-attention and the Transformer. Section 5 covers evaluation, and Section 6 lists practical considerations.

2 Encoder–Decoder Models

Let $x = (x_1, \dots, x_n)$ be a source sentence and $y = (y_1, \dots, y_m)$ its translation. A neural translation model defines the conditional probability of the output as a product of per-token terms:

$$p(y | x) = \prod_{i=1}^m p(y_i | y_{<i}, x). \quad (1)$$

The encoder turns x into a sequence of hidden states $h_1, \dots, h_n$. In recurrent models each state depends on the previous one, which makes the computation inherently sequential. Sutskever et al. [7] used deep LSTM networks and found that reversing the order of the source words made optimization noticeably easier, because it shortened the distance between the first source words and the first target words.

The decoder maintains its own state s_i and predicts y_i from s_i and a context vector c_i . Without attention, c_i is the same for every i : typically the final encoder state h_n . Every piece of information about the source must therefore flow through this one vector, no matter how long the sentence is.

3 Attention

Attention replaces the constant context with a step-specific one. At decoding step i the model assigns a score e_{ij} to every source position j , normalizes the scores with a softmax, and forms the context as a weighted average:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^n \exp(e_{ik})}, \quad c_i = \sum_{j=1}^n \alpha_{ij} h_j. \quad (2)$$

The weights α_{ij} are non-negative and sum to one, so they can be read as a soft alignment between target position i and the source. Visualizing them often reveals plausible word correspondences, although attention weights should not be taken as a complete explanation of the model’s behavior.

3.1 Additive and Multiplicative Scoring

Bahdanau et al. [1] computed the score with a small feed-forward network, an approach usually called additive attention:

$$e_{ij} = v^\top \tanh(W s_{i-1} + U h_j). \quad (3)$$

Luong et al. [5] compared several simpler alternatives, including the dot product $s_i^\top h_j$ and a bilinear form $s_i^\top W h_j$. Multiplicative scores are cheaper because they reduce to matrix products, which are highly optimized on modern hardware. They also distinguished global attention, which considers every source position, from local attention, which considers only a window around a predicted position.

3.2 Scaled Dot-Product Attention

The Transformer [8] packs the queries, keys, and values into matrices Q , K , and V and computes all attention outputs at once:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (4)$$

where d_k is the dimension of the keys. The factor $1/\sqrt{d_k}$ matters in practice. If the components of queries and keys are independent with zero mean and unit variance, their

dot product has variance d_k . For large d_k the softmax then saturates, and its gradients become extremely small. Scaling keeps the scores in a range where learning remains efficient.

Multi-head attention runs several attention functions in parallel on different learned projections of the same inputs and concatenates the results. Each head can specialize, for example in local syntax or in long-distance agreement, while the total computation stays similar to that of a single head with full dimensionality.

4 Self-Attention and the Transformer

In self-attention the queries, keys, and values all come from the same sequence, so every position can attend directly to every other position. This has two consequences. First, the number of sequential operations per layer is constant rather than proportional to the sentence length, which makes training much easier to parallelize. Second, the path between any two positions has length one, which helps the network learn long-range dependencies. The cost is quadratic in the sequence length, as summarized in Table 1.

Layer type	Per layer	Sequential	Path
Self-attention	$O(n^2d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\log_k n)$

Table 1: Complexity per layer, minimum number of sequential operations, and maximum path length for sequence length n , representation dimension d , and kernel width k , following [8].

Because attention itself ignores word order, the Transformer adds positional encodings to the input embeddings. The original model used fixed sinusoids of different frequencies, which allow the network to attend by relative position. Each layer combines attention with a position-wise feed-forward network, and every sublayer is wrapped in a residual connection [4] followed by layer normalization [3].

The decoder uses the same building blocks with two changes. Its self-attention is masked so that position i cannot see positions after i , which preserves the factorization in Eq. (1). It also contains a cross-attention sublayer whose queries come from the decoder and whose keys and values come from the encoder output. The base configuration has six encoder and six decoder layers, a model dimension of 512, and eight attention heads. A larger variant reached 28.4 BLEU on the WMT 2014 English-to-German task, which was a new state of the art at the time of publication.

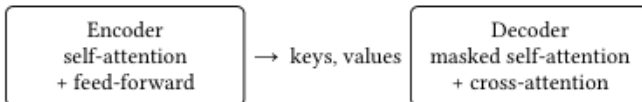


Figure 1: Information flow in an encoder–decoder Transformer. Cross-attention is the only path from the source to the target.

5 Evaluating Translations

Human judgment remains the most reliable measure of translation quality, but it is slow and expensive. Automatic metrics compare system output with one or more reference

translations. The most widely reported metric is BLEU [6], which combines modified n -gram precisions p_n for $n = 1, \dots, 4$ with a brevity penalty:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^4 w_n \log p_n \right), \quad (5)$$

where the weights are usually uniform, $w_n = 1/4$. The brevity penalty equals 1 when the candidate length c exceeds the reference length r and $\exp(1 - r/c)$ otherwise. BLEU scores depend on tokenization and on the number of references, so scores from different papers are comparable only when they were computed with the same settings.

6 Practical Considerations

Subword units. Translation vocabularies are open: names, numbers, and compounds constantly produce unseen words. Byte-pair encoding [9] splits rare words into frequent subword units, and SentencePiece [10] applies a similar idea directly to raw text without language-specific pre-tokenization.

Decoding. Greedy decoding picks the most probable token at each step, while beam search keeps several partial hypotheses. Because longer hypotheses accumulate more negative log-probability, scores are often normalized by length [11]. A larger beam does not always help; very wide beams can favor short or empty outputs.

Regularization. The Transformer was trained with label smoothing of 0.1, which hurts perplexity because the model learns to be less certain, but improves accuracy and BLEU. Dropout is applied to the output of each sublayer and to the sums of embeddings and positional encodings.

Masking. Padding tokens must be excluded from attention. In practice this means adding a large negative number to the scores of padded positions before the softmax in Eq. (4), so that their weights become effectively zero. Forgetting this mask is one of the most common sources of subtle bugs.

7 Conclusion

Attention changed neural machine translation from a model that compresses a sentence into one vector into a model that retrieves the relevant source information at every step. The same mechanism, applied within a sequence, became the foundation of the Transformer. Readers who implement these models should pay particular attention to scaling, masking, and consistent evaluation, since small differences in these details can change results considerably.

References

- [1] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.
- [2] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *EMNLP*, 2014.
- [3] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016.
- [4] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.